메타 플밍 13기

3D Action Game

프로젝트 보고서

발표자: 최준영

DEVELROCKET

발표 내용 요약

CONTENTS

01 | 기획 의도 — 장르 선택의 이유 및 개발 컨셉

02 | 주요 구현 내용 — 구현한 기능 중 주요기능의 간략한 설명

03 | 주요 기술 설명 — 구현한 기능의 기술적 설명

04 | 개발 결과 자가평가 및 시연 — 프로젝트의 달성률 및 과정과 결과를 평가하고 게임 시연

05 | 질의 응답 — 질의응답

01

기획의도

게임 소개와 장르 선택의 이유 및 개발 컨셉

게임 소개

스태미나 기반의 전략적인 타격과 회피, 파밍과 성장 그리고 거리에 따라
실시간으로 판단하는 보스와의 치열한 전투를 담아낸
3인칭 3D 액션 게임



사냥 & 파밍

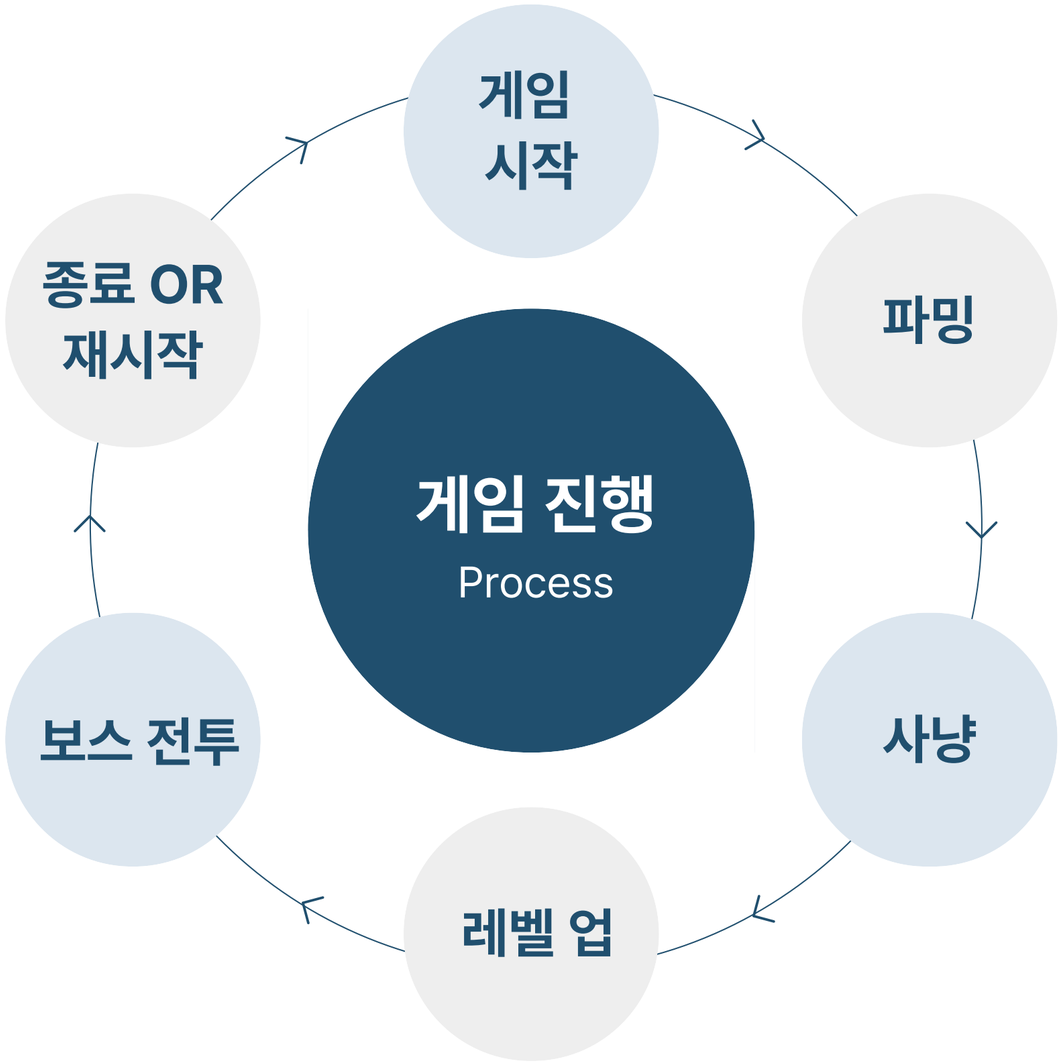


보스 전투



시작 화면

게임 진행 구조



기획 의도

개발 의도-3인칭 3D Action을 선택한 이유

- 프로젝트를 3D 액션 장르로 선택한 가장 큰 이유는 플레이어의 입력에 즉각적으로 반응하며 다양한 조작이 가능한 '플레이어의 정밀한 조작감'을 직접 설계하고 구현해 보고 싶었기 때문입니다.

- 타격 판정, 프레임 단위의 입력 처리, 그리고 애니메이션 캔슬 등 액션 게임의 뼈대가 되는 핵심 로직들을 깊이 있게 다루고 학습하는 데 있어, 묵직하고 정교한 전투를 지향하는 소울류 스타일의 장르가 최적의 방향성이 될 것이라고 판단했습니다.

개발 컨셉- Dark Souls 따라 만들기

- 다크소울의 핵심 재미 요소를 분석하고, 이를 시스템적으로 최대한 비슷하게 재현하는 것을 최우선 목표로 삼았습니다. 스태미나 자원 관리를 강제하는 전투와 무적 프레임을 통한 회피로 긴장감 넘치는 전투를 구현했으며, 상태 머신(FSM)을 활용해 플레이어와의 거리 및 쿨타임에 따라 합리적인 패턴을 스스로 선택하는 지능적인 보스 AI를 설계했습니다.

- 또한 화톳불 리스폰, 물약 섭취, 스탯 기반의 레벨업 등 원작의 코어 시스템을 최대한 담아내려 하였습니다.

02

주요 구현 내용

구현한 기능 중 주요기능의 간략한 설명

Player 조작 및 전투 시스템

선입력 및 스태미나 기반의 액션

공격, 구르기, 달리기 등 모든 주요 행동이 스태미나를 소모하도록 하여 전략적인 자원 관리가 필요한 전투를 구현했으며, 무적 판정이 있는 구르기(회피)와 연속 공격을 선입력 시스템을 사용하여 각 모션의 연결을 부드럽게 이어지도록 만들었습니다.

성장 시스템

필드의 몬스터를 사냥하여 돈을 얻을 수 있고 세이브 포인트에서 돈을 사용하여 HP, STR, DEX등 스텟을 강화할 수 있으며, 필드에서 장비 아이템을 루팅하여 각 장비 아이템을 각 슬롯에 장착하여 성장할 수 있다.

타겟팅(락온) 및 시점 제어

반경 내 적을 탐지하여 시점을 고정하는 락온 시스템을 구현했으며, 다수의 적과 싸울 때 마우스 조작으로 자연스럽게 타겟을 전환할 수 있습니다.

아이템

소비 아이템을 구현하여 전투중 물약을 사용하여 회복을 할 수있고, 장비 아이템을 파밍하여 무기, 머리, 상의, 하의, 신발, 장신구, 소비템 의 구분에 따라 각 장비 슬롯에 장착할 수 있다.

필드 몹과의 전투 및 보스전



일반 적

Idle상태에서 시야각과 장애물을 판별하여 일정 범위 내의 플레이어를 인식하여 랜덤 패턴 중 한가지의 공격을 하고, 공격 쿨타임 중에는 무작위 돌진이 아닌 거리를 유지하며 주변을 맴도는 (게걸음/Strafe) 견제 행동을 합니다.

행동 패턴: 플레이어 감지 → 플레이어에게 전진 → 공격 → 공격 쿨타임이 돌아올때 까지 견제



보스

플레이어와의 거리와 각도, 스킬별 쿨타임 등을 계산하여 가장 적절한 공격 패턴을 스스로 선택하며, 보스의 체력이 특정 수치 이하(50%)로 떨어지면 포효 연출과 함께 더욱 강력한 공격 패턴들을 구사하는 2페이즈로 돌입합니다.

행동 패턴: 플레이어 감지 → 플레이어와의 거리와 각도 및 페이즈등을 반영하여 랜덤 패턴 동작

UI 구현

- 인벤토리 : 아이템을 무기, 방어구, 장신구 등으로 분류하여 상세 정보를 확인할 수 있도록 구성했습니다

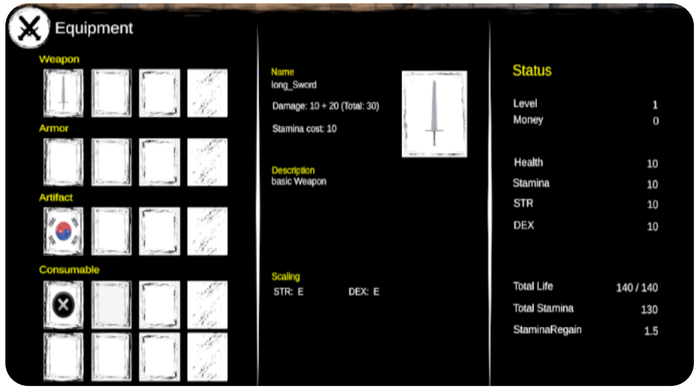
- 장비 창 : 장비를 슬롯에 자유롭게 장착할 수 있으며, 장비에 적용 되는 플레이어의 상세 스탯과 정보를 한눈에 파악할 수 있습니다

- 인게임 UI : 체력 및 스태미나 바를 상단에 배치하고, 하단에는 현재 장착된 무기와 물약을 보여주는 퀵슬롯을 배치했습니다.

- 강화(레벨업) 시스템 UI : 재화를 소모해 스탯을 투자할 때 상승하는 능력치와 소모 비용을 직관적인 미리보기로 제공하여, 전략적으로 캐릭터를 육성할 수 있도록 설계했습니다.



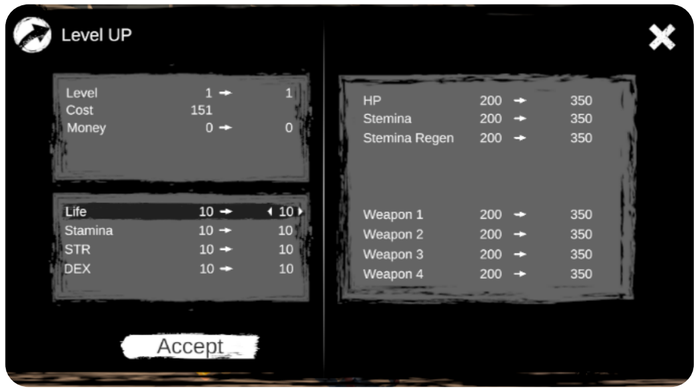
인벤토리



장비 창



인게임 UI



강화 시스템 UI

03

주요 기술 설명

구현한 기능의 구체적인 기술적 설명

플레이어 및 적 조작 - FSM

Player의 FSM (입력 중심)

플레이어의 키보드/마우스 '입력(Input)'을 최우선으로 감지하여 상태를 즉각적으로 전환하는 데 집중했습니다.

중점 - 플레이어의 조작 입력, 애니메이션 진행도(선입력 및 캔슬 타이밍), 스태미나 잔여량

Enemy의 FSM (거리 중심)

일반 적은 연산 비용을 줄이기 위해 플레이어와의 거리 및 시야를 기준으로 추적(Chase), 견제(Strafe), 타격 상태를 순차적으로 오가도록 가볍게 설계했습니다.

중점 - 플레이어 감지 여부, 플레이어의 거리, 공격 쿨타임

Boss의 FSM (데이터 주도형 의사결정)

보스는 스스로 다음 행동을 결정하는 '판단(Decision) 상태'를 별도로 가집니다. 현재 거리, 개별 스킬 쿨타임, 페이즈 조건, 그리고 '패턴별 가중치(Weight)'를 조합한 룰렛 시스템을 돌려 예측 불가능하면서도 합리적인 AI를 구현했습니다.

중점 - 플레이어와의 거리, 개별 패턴 쿨타임, 보스의 현재 페이즈, 패턴 가중치(확률)

Decision State

- 필터링(조건 검사)

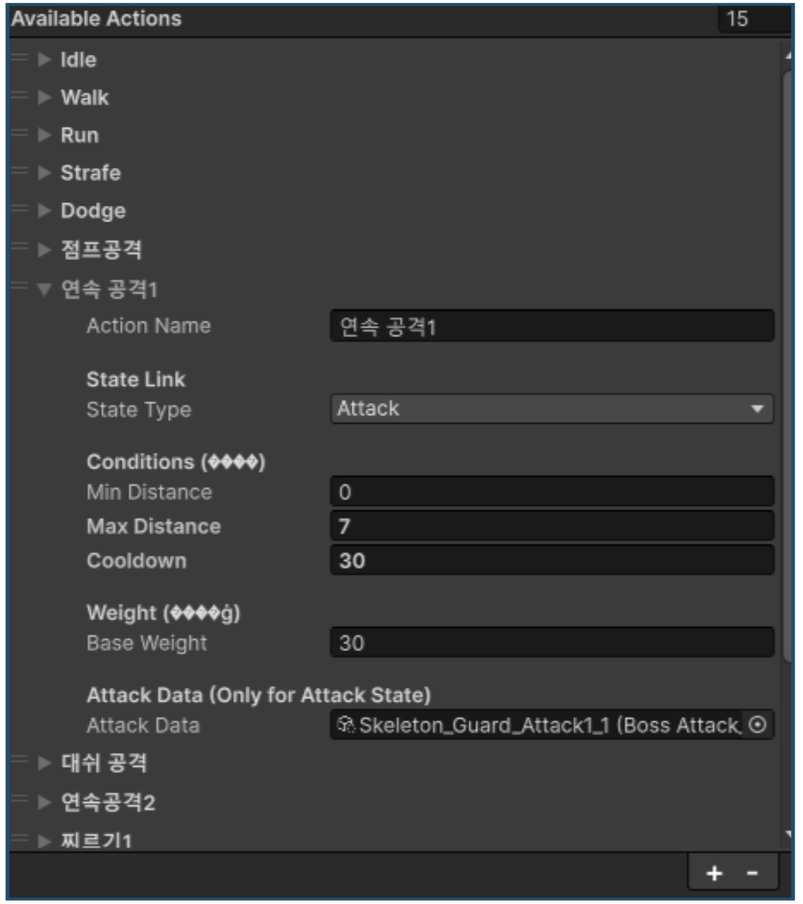
보스는 행동을 결정하기 전, 자신이 가진 여러 패턴 중 '현재 쓸 수 없는 것'들을 걸러냅니다. 현재 플레이어와의 거리, 개별 패턴의 남은 쿨타임, 그리고 보스의 현재 페이즈를 실시간으로 계산하여 조건에 맞지 않는 행동은 후보에서 완전히 제외합니다.

- 가중치(Weight) 기반의 룰렛 시스템

조건을 통과한 유효한 패턴들 중에서 다음 행동을 고를 때는 단순한 랜덤이 아니라 가중치(baseWeight)를 활용합니다.

이러한 의사결정 시스템을 통해 보스는 현재 거리나 페이즈에 맞는 '합리적인 선택'을 하면서도, 가중치 룰렛을 통한 '무작위성'을 동시에 가지게 됩니다.

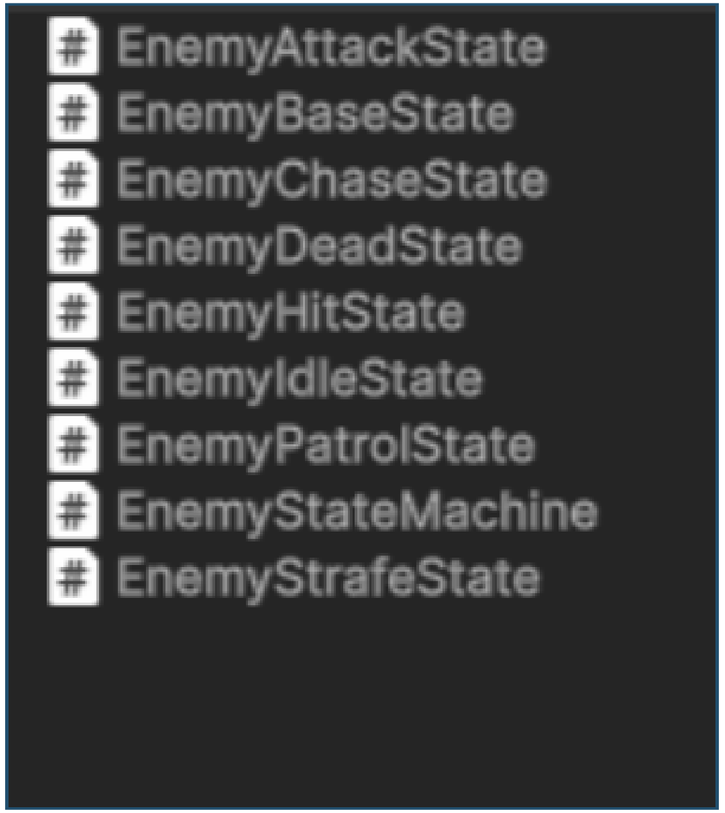
ex) 근거리에서 발동 가능한 패턴 A의 가중치가 70이고 패턴 B의 가중치가 30이라면, 보스는 70%의 확률로 A를, 30%의 확률로 B를 선택하게 됩니다.



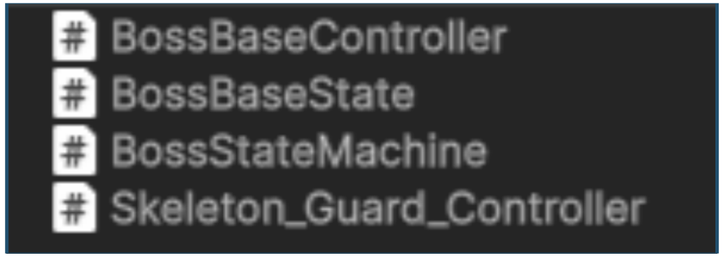
각 FSM의 State 구조

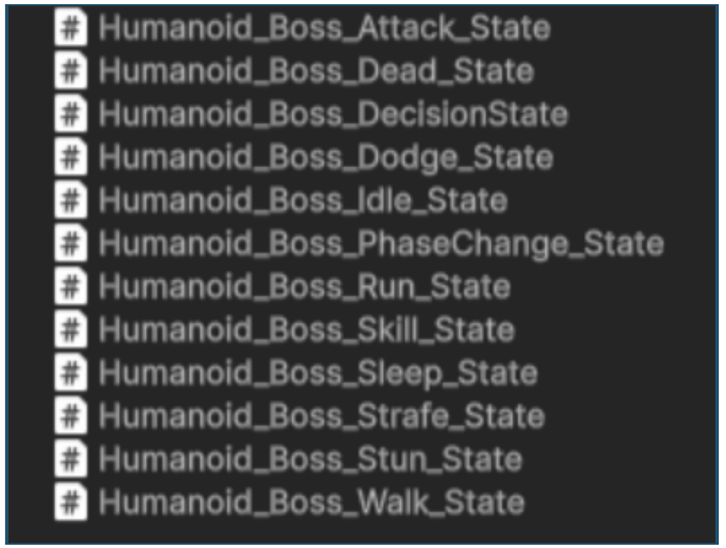


Player



Enemy





Boss

정교한 애니메이션 제어 (GetCurrentAnimatorStateInfo)

- GetCurrentAnimatorStateInfo : 현재 재생 중인 애니메이션의 상태 정보를 가져오는 함수

- GetCurrentAnimatorStateInfo의 normalizedTime라는 속성을 사용하여 애니매이션의 진행도를 0~1사이의 값으로 받아 애니메이션의 시각적 흐름과 실제 코드의 논리적 판정(데미지가 들어가는 타이밍 등)을 완벽하게 동기화하기 위해 사용했습니다.

- 활용 방식 : 애니메이션의 진행도를 실시간으로 추적하여, 정밀한 타격 판정(Hitbox) 온오프, 선입력을 활용한 콤보 및 캔슬 시스템, 구르기 무적 프레임 구현 등 애니매이션의 타이밍을 맞추어야하는 이벤트들을 구현하는데 사용하였습니다.



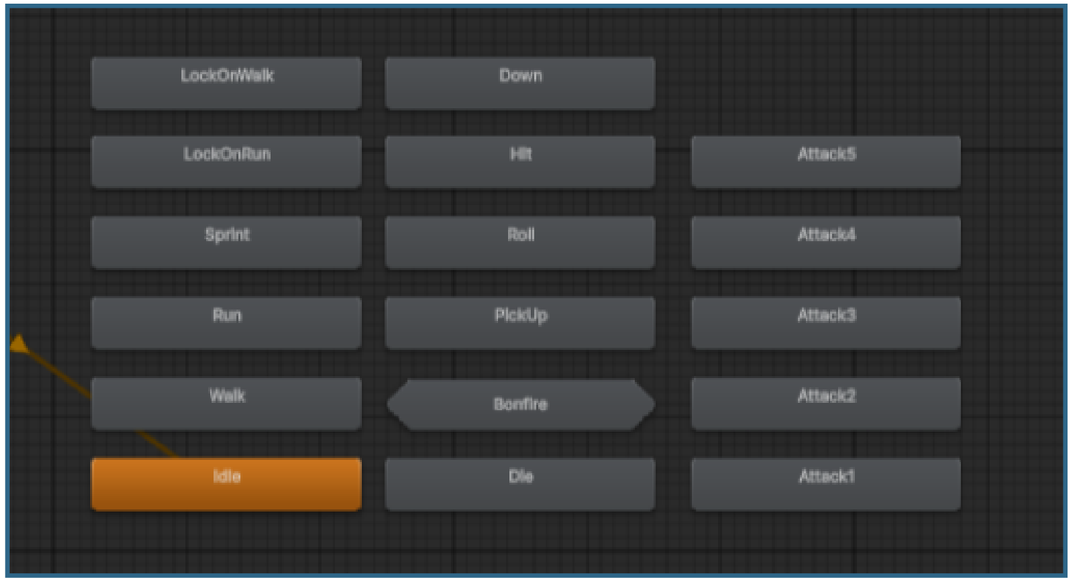
롱소드 Attack SO Data

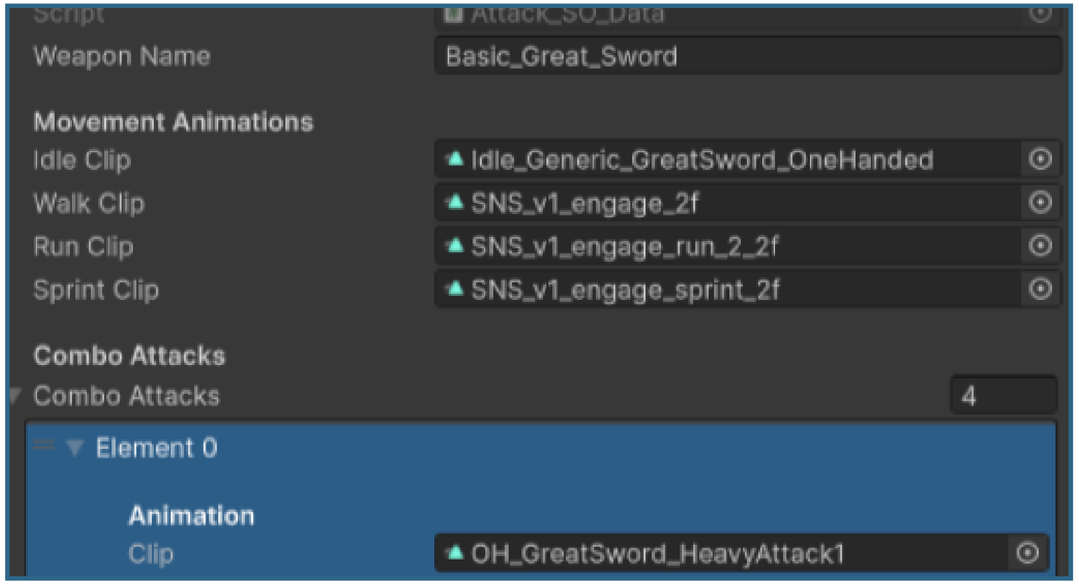
유연한 무기, 패턴 변경 시스템(확장성) (AnimatorOverrideController & SO)

- AnimatorOverrideController : 애니메이터의 구조를 그대로 유지한 채, 런타임에 들어가는 '애니메이션 클립'만 갈아 끼울 수 있게 해주는 기능입니다.

- 활용 방식: 플레이어가 장검에서 대검으로 무기를 바꾸거나, 적이 다른 패턴을 구사할 때 새로운 애니메이터 노드를 만들 필요 없이 노드 내부에 들어갈 클립만 동적으로 교체했습니다.

- Scriptable Object(SO)와 결합하여, 공격 모션, 데미지 배율, 판정 타이밍이 하나로 묶인 '무기/패턴 데이터'를 만들어 기획 확장이 매우 용이한 데이터 주도형 아키텍처를 구축했습니다.





UI 시스템 설계 - AI사용

Action(이벤트) 기반 업데이트

- Action? - 특정 조건이 충족되었을 때 등록된 함수들을 일제히 실행시켜 주는 일종의 '방송국' 역할
- 체력이나 재화가 매 프레임 업데이트되는 것을 막기 위해, 수치가 변동될 때만 UI를 갱신하도록 Action(이벤트)을 활용한 옵저버 패턴을 적용했습니다.

LINQ를 활용한 동적 인벤토리 필터링

- LINQ? - 리스트나 배열 같은 데이터 무리에서 내가 원하는 조건의 데이터만 뽑아낼 수 있게 해주는 기능
- 무기, 방어구, 소모품을 별도의 리스트로 쪼개어 관리하지 않고, InventoryManager에 단 하나의 마스터 명부(allItems)를 두었습니다.
- 아이템을 습득하거나 장비창에서 특정 카테고리를 열 때, C#의 LINQ(Where, FirstOrDefault)를 활용하여 필요한 데이터만 동적으로 필터링하고 검색했습니다. 이를 통해 데이터 중복과 복잡한 반복문을 제거하고, 추후 아이템 타입이 늘어나더라도 코드를 수정할 필요 없는 높은 유지보수성을 확보했습니다.

문제 해결 - (root motion 이동량 문제, 몬스터의 플레이어 감지)

OnAnimatorMove를 활용한 하이브리드 루트 모션

OnAnimatorMove - Root Motion이 물리 엔진에 적용되기 직전, 코드로 이를 재가공할 수 있게 해주는 콜백 함수

무작정 애니메이션의 이동값을 따르지 않고, 타겟과의 거리를 실시간으로 계산하여 너무 가까우면 이동량(forward)을 감쇄시켜 캐릭터가 겹치며 덜덜거리는 현상을 방지하였습니다.

시야각 구현 및 방해물 구현

OverlapSphere - 지정된 반경 내의 모든 객체를 탐지하는 구형 물리 함수

Raycast - 보이지 않는 광선을 발사해 충돌 여부를 판별하는 선형 물리 함수

Dot - 두 방향 벡터 사이의 각도를 수학적으로 계산하는 함수

무작정 일정 거리의 플레이어를 감지하는 것이 아니라, 우선 OverlapSphere로 대상을 탐지한 뒤, 지정한 시야각 내에 있는지 판별합니다. 이후 Raycast를 발사해 대상과 나 사이에 벽(장애물)이 없는지 최종 검증하여, 벽 너머의 적은 락온/인식하지 않는 현실적인 시야 시스템을 구축했습니다.

04

개발 결과 자가평가 및 시연

프로젝트의 달성률 및 과정과 결과를 평가하고 게임 시연

결과 분석

기획 당시 개발 목표

1. FSM을 통한 플레이어 조작 구현
2. 인벤토리 및 장비 장착 시스템 구현
3. 패드 및 키보드/마우스 완벽 대응
4. 회피(Roll)와 대시(Dash) 구현, 타겟 락온(Lock-on) 시스템 구현
5. 체크포인트(화톳불 시스템)를 통한 휴식 및 부활 구현
6. 몬스터 및 보스 AI 구현 및 보스 페이즈 시스템 구현
7. 다양한 형태의 보스 구현(비인간형)
8. 히트박스 기반의 정교한 타격/피격 판정 구현
9. Auto 세이브 기능 구현

개발 진행도

1. FSM을 통한 플레이어 조작 구현 ✓
2. 인벤토리 및 장비 장착 시스템 구현 ✓
3. 패드 및 키보드/마우스 완벽 대응 ✗
4. 회피(Roll)와 대시(Dash) 구현, 타겟 락온(Lock-on) 시스템 구현 ✓
5. 체크포인트(화톳불 시스템)를 통한 휴식 및 부활 구현 ✓
6. 몬스터 및 보스 AI 구현 및 보스 페이즈 시스템 구현 ✓
7. 다양한 형태의 보스 구현(비인간형) ✗
8. 히트박스 기반의 정교한 타격/피격 판정 구현 ✓
9. Auto 세이브 기능 구현 ✗
10. 장착 무기마다 개별적인 공격, 움직임 애니메이션 적용 +
11. 재화를 사용하여 레벨업하는 시스템 구현 +

자가 평가

이번 프로젝트를 통해 느낀 점

아직 모르는 Unity 함수와 기술들이 많아서 AI와 검색의 도움을 많이 받았던 것 같고 AI가 확실히 빠른 문제 해결에는 매우 큰 도움이 되었으나 스스로 부족하다는것을 뼈져리게 느꼈던것 같습니다. AI의 도움을 받기 했지만 이번 프로젝트로 다양한 기능들을 직접 구현할 수 있었고, 이번 경험을 통해 부족한 부분을 명확히 알게 된 만큼 앞으로 더 열심히 배운다면 훨씬 완성도 높은 결과물을 만들 수 있다는 확신을 얻었습니다.

다음 프로젝트 목표

단순히 기능이 동작하는 것에 그치지 않고 유지보수나 최적화의 측면을 고려하여 성능 최적화가 되어있는 프로젝트를 만들어 보고 싶습니다

05

질의 응답